

Finance Tools How-To: Stocks, Shares, and Currency Data

This guide explains how to use ragent’s built-in finance tools to access real-time stock quotes, historical price charts, company fundamentals, options chains, analyst recommendations, symbol search, and currency exchange rates from within an agent session, the TUI, or the HTTP server.

Purpose

ragent ships with a provider-agnostic finance toolset that lets an agent answer questions about financial markets without leaving the session. The tools are registered under the `network:fetch` permission category and are available in every agent preset by default.

The toolset is designed around a single `FinanceProvider` trait with three concrete adapters:

- **Yahoo Finance** (free, no API key required) - the default provider.
- **Alpha Vantage** (paid, requires an API key) - configured via `ragent.json`.
- **TwelveData** (paid, requires an API key) - configured via `ragent.json`.

When a paid provider is configured, it takes priority for all finance calls. If the paid provider does not implement a specific endpoint (for example, Alpha Vantage has no options-chain endpoint), ragent can optionally fall back to the free Yahoo adapter so the tool still returns data instead of an error.

Fallback scope: the endpoint fallback applies to `stock_fundamentals` and `stock_recommendations` only. `stock_quote` and `stock_history` have a separate fallback that triggers only on “symbol missing or invalid” errors. `stock_options`, `stock_search`, and the `currency_*` tools always surface paid-provider errors. Every finance tool publishes an `AgentNotice` naming the provider that served the call (visible in the TUI log panel), and symbols/base/quote inputs are uppercased before provider calls.

Architecture Overview

The finance module lives in `crates/ragent-tools-extended/src/finance/` and is organised as follows:

```
finance/
  mod.rs          -- Public re-exports
  model.rs        -- Provider-agnostic data types (Quote, OhlcvBar, ...)
  provider.rs     -- The FinanceProvider trait
  error.rs        -- Normalized FinanceError enum
  cache.rs        -- In-memory quote cache (60s TTL by default)
  rate_limit.rs   -- Per-provider rate limiter with exponential backoff
  throttle.rs     -- Minimum-interval throttle shared across providers
  providers/
    mod.rs        -- Provider module re-exports
    paid.rs       -- PaidProvider router (Alpha Vantage) + Yahoo cache
    yahoo.rs      -- YahooFinanceProvider (free, via yfinance_rs)
    twelvedata.rs -- TwelveDataProvider (paid)
    yahoo/
      normalize.rs -- Yahoo error normalization helpers
  tools/
    mod.rs        -- Tool registration helpers + with_yahoo_fallback
    quote.rs      -- stock_quote
```

```

history.rs          -- stock_history
fundamentals.rs     -- stock_fundamentals
search.rs          -- stock_search
options.rs         -- stock_options
recommendations.rs -- stock_recommendations
currency_rate.rs   -- currency_rate
currency_history.rs -- currency_history

```

The FinanceProvider Trait

Every adapter implements the same async trait, defined in `crates/ragent-tools-extended/src/finance/provider.rs`:

```

#[async_trait::async_trait]
pub trait FinanceProvider: Send + Sync + std::fmt::Debug {
    fn name(&self) -> &str;
    fn is_available(&self) -> bool;

    async fn quote(&self, symbol: &str) -> FinanceResult<Quote>;
    async fn history(&self, symbol: &str, interval: &str, period: &str)
        -> FinanceResult<Vec<OhlcvBar>>;
    async fn fundamentals(&self, symbol: &str) -> FinanceResult<Fundamentals>;
    async fn currency_rate(&self, base: &str, quote: &str) -> FinanceResult<CurrencyRate>;
    async fn currency_history(&self, base: &str, quote: &str, interval: &str, period: &str)
        -> FinanceResult<Vec<OhlcvBar>>;
    async fn search(&self, query: &str) -> FinanceResult<Vec<SearchResult>>;
    async fn options(&self, symbol: &str, expiration: Option<&str>)
        -> FinanceResult<Vec<OptionContract>>;
    async fn recommendations(&self, symbol: &str) -> FinanceResult<Vec<RecommendationPeriod>>;
}

```

This means every tool call goes through the same code path regardless of which provider is active. The tool layer selects the provider via `default_provider(config)`, calls the appropriate trait method, and serializes the result as pretty-printed JSON.

Provider Selection Logic

The function `default_provider()` in `crates/ragent-tools-extended/src/finance/providers/paid.rs` implements the selection chain:

1. If the config's `finance.provider` field is not "yahoo" and an `api_key` is present (`is_paid_provider_configured()` returns true), the corresponding paid adapter is constructed and returned.
2. If construction of the paid provider fails (for example, a malformed base URL), ragent logs a warning and falls back to Yahoo.
3. Otherwise, a cached `YahooFinanceProvider` is returned. The Yahoo provider is cached per effective configuration (User-Agent + requests-per-minute) so that all finance tools share a single `YfClient`, cookie/crumb state, rate-limit backoff, and throttle.

Yahoo Fallback

Only `stock_fundamentals` and `stock_recommendations` use the `with_yahoo_fallback` helper. This helper calls the paid provider first. If the paid provider returns a `ProviderFailure` whose mes-

sage indicates the endpoint is “not implemented”, “does not exist”, or “not available on your plan”, and the `yahoo_fallback` config flag is enabled, the helper transparently retries the call against the free Yahoo adapter. This lets you mix a paid provider for quotes/history with Yahoo for fundamentals and recommendations without changing any tool calls.

`stock_quote` and `stock_history` carry a narrower inline fallback that fires only when the paid provider reports the *symbol* as missing or invalid. `stock_options`, `stock_search`, `stock_recommendations` (direct call), and the `currency_*` tools have **no** fallback — a configured paid provider’s errors (including its not-implemented options stub) surface as tool errors even with `yahoo_fallback: true`.

The `yahoo_fallback` flag defaults to: - **Enabled** when no paid provider is configured (Yahoo is the only provider, so “fallback” is a no-op). - **Disabled** when a paid provider is explicitly configured, so that paid-provider errors are surfaced clearly instead of being masked by Yahoo rate-limit messages. Set `"yahoo_fallback": true` in the finance config block to enable cross-provider fallback.

If a configured paid provider cannot even be constructed (for example a malformed `base_url`), `default_provider` logs a warning and falls back to Yahoo silently — config mistakes degrade quietly rather than erroring.

Configuration

Finance provider settings are read from the `finance` block in `ragent.json` (or `ragent.jsonc`). The config type is `FinanceProviderConfig`, defined in `crates/ragent-config/src/finance.rs`.

Default (Free Yahoo, No Configuration Needed)

If you do not include a `finance` block at all, `ragent` uses the free Yahoo Finance adapter with sensible defaults:

```
{
  // No "finance" block - Yahoo Finance is used automatically
}
```

Alpha Vantage

```
{
  "finance": {
    "provider": "alpha_vantage",
    "api_key": "YOUR_ALPHA_VANTAGE_KEY",
    "min_call_interval_seconds": 5,
    "yahoo_fallback": true
  }
}
```

TwelveData

```
{
  "finance": {
    "provider": "twelvedata",
    "api_key": "YOUR_TWELVEDATA_KEY",
    "base_url": "https://api.twelvedata.com",
    "yahoo_fallback": true
  }
}
```

```

    }
}

```

All Config Fields

Field	Type	Default	Description
provider	string	"yahoo"	Provider name: "yahoo", "alpha_vantage", or "twelvedata".
api_key	string?	null	API key for paid providers. Required when provider is not "yahoo".
base_url	string?	null	Override the provider's API base URL. Useful for proxies or self-hosted endpoints.
requests_per_minute	u32?	null	Yahoo-only throttle target. When set, Yahoo calls are spaced at least 60 / rpm seconds apart.
user_agent	string?	null	Custom User-Agent header for the free Yahoo provider.
min_call_interval_seconds	u64	5	Minimum seconds between any two finance API calls. Shared across Yahoo and paid providers.
yahoo_fallback	bool?	null	When true, fall back to Yahoo if the paid provider fails on a specific endpoint. Defaults to true for Yahoo-only, false when a paid provider is configured.

Available Tools

There are eight finance tools, each exposed as a registered tool that an agent can invoke directly. All tools accept JSON input and return pretty-printed JSON output.

stock_quote

Fetch the latest market quote for a single ticker symbol.

Parameters:

Parameter	Type	Required	Description
symbol	string	Yes	Ticker symbol, e.g. "AAPL".

Returns: A Quote object (see data model below).

Example - TUI / agent prompt:

What is the current price of Apple stock?

The agent will call stock_quote with {"symbol": "AAPL"} and receive:

```
{
  "symbol": "AAPL",
  "price": 229.87,
  "open": 228.50,
  "high": 230.41,
  "low": 227.62,
  "close": 229.87,
  "volume": 45231800,
  "change": 1.37,
  "change_percent": 0.6,
  "currency": "USD",
  "market_state": "REGULAR",
  "timestamp": "2026-08-22T14:30:00Z"
}
```

The stock_quote tool also uses an in-memory QuoteCache with a 60-second TTL. Repeated calls for the same (provider, symbol) pair within 60 seconds return the cached quote without hitting the provider API. The metadata field "cached": true indicates a cache hit.

stock_history

Fetch historical OHLCV (open, high, low, close, volume) bars for a ticker, suitable for charting or back-testing.

Parameters:

Parameter	Type	Required	Default	Description
symbol	string	Yes	-	Ticker symbol.

Parameter	Type	Required	Default	Description
interval	string	No	"1d"	Candle interval: "1d" (daily), "1wk" (weekly), or "1mo" (monthly).
period	string	No	"1mo"	Lookback period. Supported values depend on the provider. Yahoo accepts: "1d", "5d", "1w", "1mo", "3mo", "6mo", "1y", "5y", "max". Short periods (1d, 5d, 1w) are rounded up to the smallest supported Yahoo range (1 month). TwelveData and Alpha Vantage compute a date cutoff from the period.

Returns: An array of OhlcvBar objects sorted by timestamp ascending.

Example - 1-year daily history for MSFT:

Show me MSFT's daily price history for the past year.

The agent calls `stock_history` with:

```
{
  "symbol": "MSFT",
  "interval": "1d",
  "period": "1y"
}
```

Sample response (truncated):

```
[
  {
    "timestamp": "2025-08-25T13:30:00Z",
    "open": 420.12,
    "high": 425.50,
    "low": 419.01,
    "close": 423.88,
    "volume": 12345678
  },
  {
    "timestamp": "2025-08-26T13:30:00Z",
    "open": 423.90,
```

```

    "high": 428.00,
    "low": 422.15,
    "close": 427.30,
    "volume": 11234567
  }
]

```

Example - Weekly bars for 6 months:

```

{
  "symbol": "GOOGL",
  "interval": "1wk",
  "period": "6mo"
}

```

stock_fundamentals

Fetch key fundamental metrics for a company: market cap, P/E ratios, EPS, dividend yield, 52-week range, and sector classification.

Parameters:

Parameter	Type	Required	Description
symbol	string	Yes	Ticker symbol.

Returns: A Fundamentals object.

Example:

What are the fundamentals for Tesla?

The agent calls stock_fundamentals with {"symbol": "TSLA"}:

```

{
  "symbol": "TSLA",
  "name": "Tesla, Inc.",
  "sector": "Consumer Cyclical",
  "market_cap": 783456789012,
  "trailing_pe": 65.4,
  "forward_pe": 58.2,
  "eps": 3.51,
  "dividend_yield": null,
  "fifty_two_week_high": 314.67,
  "fifty_two_week_low": 182.63
}

```

Note: TwelveData fundamentals require the /statistics endpoint, which is only available on paid TwelveData plans. On the free TwelveData tier this endpoint returns an error, and ragent will fall back to Yahoo if yahoo_fallback is enabled.

stock_search

Search for ticker symbols matching a company or asset name fragment. Useful when you know the company name but not the exact ticker.

Parameters:

Parameter	Type	Required	Description
query	string	Yes	Company or asset name to search for.

Returns: An array of SearchResult objects.

Example:

What is the ticker symbol for NVIDIA?

The agent calls `stock_search` with `{"query": "NVIDIA"}`:

```
[
  {
    "symbol": "NVDA",
    "name": "NVIDIA Corporation",
    "exchange": "NMS",
    "asset_class": "equity"
  }
]
```

stock_options

Fetch the options chain (calls and puts) for a ticker, optionally filtered by a specific expiration date.

Parameters:

Parameter	Type	Required	Description
symbol	string	Yes	Ticker symbol.
expiration	string	No	Optional expiration date in YYYY-MM-DD format. If omitted, the nearest expiration is returned.

Returns: An array of OptionContract objects.

Example:

Show me the options chain for SPY expiring 2026-09-19.

The agent calls `stock_options` with:

```
{
  "symbol": "SPY",
  "expiration": "2026-09-19"
}
```


Sample response (truncated):

```
[
  {
    "strike": 450.0,
    "expiration": "2026-09-19",
    "kind": "Call",
    "last_price": 12.50,
    "bid": 12.30,
    "ask": 12.70,
    "volume": 5234,
    "open_interest": 18234,
    "implied_volatility": 0.18
  },
  {
    "strike": 450.0,
    "expiration": "2026-09-19",
    "kind": "Put",
    "last_price": 3.20,
    "bid": 3.10,
    "ask": 3.30,
    "volume": 3210,
    "open_interest": 15234,
    "implied_volatility": 0.19
  }
]
```

Note: Options chains are only implemented for the Yahoo provider. Alpha Vantage and TwelveData return a `ProviderFailure` for this endpoint. With `yahoo_fallback` enabled, the tool transparently falls back to Yahoo.

stock_recommendations

Fetch analyst recommendation trend counts (strong buy, buy, hold, sell, strong sell) for recent reporting periods.

Parameters:

Parameter	Type	Required	Description
symbol	string	Yes	Ticker symbol.

Returns: An array of `RecommendationPeriod` objects.

Example:

What do analysts recommend for Amazon stock?

The agent calls `stock_recommendations` with `{"symbol": "AMZN"}`:

```
[
  {
```

```

    "period": "0m",
    "strong_buy": 28,
    "buy": 15,
    "hold": 8,
    "sell": 1,
    "strong_sell": 0
  },
  {
    "period": "-1m",
    "strong_buy": 25,
    "buy": 17,
    "hold": 10,
    "sell": 2,
    "strong_sell": 0
  }
]

```

currency_rate

Fetch the current exchange rate between two currencies.

Parameters:

Parameter	Type	Required	Description
base	string	Yes	Source currency code, e.g. "USD".
quote	string	Yes	Target currency code, e.g. "EUR".

Returns: A CurrencyRate object.

Example:

What is the current USD to EUR exchange rate?

The agent calls currency_rate with {"base": "USD", "quote": "EUR"}:

```

{
  "base": "USD",
  "quote": "EUR",
  "rate": 0.8523,
  "timestamp": "2026-08-22T14:30:00Z"
}

```

currency_history

Fetch historical OHLCV bars for a currency pair, suitable for FX charting.

Parameters:

Parameter	Type	Required	Default	Description
base	string	Yes	-	Source currency code.

Parameter	Type	Required	Default	Description
quote	string	Yes	-	Target currency code.
interval	string	No	"1d"	Candle interval: "1d", "1wk", or "1mo".
period	string	No	"1mo"	Lookback period (same values as stock_history).

Returns: An array of OhlcvBar objects.

Example - GBP/JPY monthly history for 1 year:

Show me the GBP to JPY monthly exchange rate history for the past year.

The agent calls currency_history with:

```
{
  "base": "GBP",
  "quote": "JPY",
  "interval": "1mo",
  "period": "1y"
}
```

Data Models

All data types are defined in `crates/ragent-tools-extended/src/finance/model.rs` and are serialized as JSON by every tool.

Quote

Field	Type	Description
symbol	string	Ticker symbol.
price	f64	Latest traded price.
open	f64	Session open price.
high	f64	Session high.
low	f64	Session low.
close	f64	Current/close price.
volume	u64	Session volume.
change	f64	Absolute change from previous close.
change_percent	f64	Percentage change from previous close.
currency	string	Quote currency code (e.g. "USD").
market_state	string	Market state (e.g. "REGULAR", "CLOSED").
timestamp	DateTime	UTC timestamp of the quote.

OhlcvBar

Field	Type	Description
timestamp	DateTime	UTC timestamp of the bar.
open	f64	Bar open price.
high	f64	Bar high price.
low	f64	Bar low price.
close	f64	Bar close price.
volume	u64	Bar volume.

Fundamentals

Field	Type	Description
symbol	string	Ticker symbol.
name	string?	Company name.
sector	string?	Sector classification.
market_cap	u64?	Market capitalization.
trailing_pe	f64?	Trailing P/E ratio.
forward_pe	f64?	Forward P/E ratio.
eps	f64?	Earnings per share.
dividend_yield	f64?	Dividend yield (percentage).
fifty_two_week_high	f64?	52-week high.
fifty_two_week_low	f64?	52-week low.

CurrencyRate

Field	Type	Description
base	string	Source currency code.
quote	string	Target currency code.
rate	f64	Exchange rate.
timestamp	DateTime	UTC timestamp.

OptionContract

Field	Type	Description
strike	f64	Strike price.
expiration	NaiveDate	Expiration date.
kind	enum	"Call" or "Put".
last_price	f64	Last traded price.
bid	f64	Bid price.
ask	f64	Ask price.
volume	u64	Volume.
open_interest	u64	Open interest.

Field	Type	Description
implied_volatility	f64?	Implied volatility.

SearchResult

Field	Type	Description
symbol	string	Ticker symbol.
name	string?	Company or asset name.
exchange	string?	Exchange code.
asset_class	string?	Asset class.

RecommendationPeriod

Field	Type	Description
period	string	Reporting period label (e.g. "0m", "-1m").
strong_buy	u32	Count of strong buy ratings.
buy	u32	Count of buy ratings.
hold	u32	Count of hold ratings.
sell	u32	Count of sell ratings.
strong_sell	u32	Count of strong sell ratings.

Provider Feature Matrix

The table below shows which functions are implemented (native) by each provider. Cells marked “Fall-back” indicate that the paid provider does not implement the endpoint natively, but ragent will transparently fall back to the free Yahoo adapter when yahoo_fallback is enabled in the config.

Function	Yahoo (free)	Alpha Vantage (paid)	TwelveData (paid)
stock_quote	Native	Native	Native
stock_history	Native	Native (daily only)	Native
stock_fundamentals	Native	Not implemented	Native (paid plan)
stock_search	Native	Not implemented	Not implemented
stock_options	Native	Not implemented	Not implemented
stock_recommendations	Native	Native	Not implemented
currency_rate	Native	Not implemented	Not implemented
currency_history	Native	Not implemented	Not implemented

Provider Notes

Yahoo Finance (free): - No API key required. Uses the yfinance_rs crate which scrapes the public Yahoo Finance endpoints. - Subject to Yahoo’s rate limits. The built-in RateLimiter uses exponential backoff when a 429 or 999 response is received. - The requests_per_minute config field adds a client-side throttle that spaces requests at least 60 / rpm seconds apart. - All eight endpoints are implemented natively.

Alpha Vantage (paid): - Implements quote (via GLOBAL_QUOTE), history (via TIME_SERIES_DAILY with compact output), and recommendations (via RECOMMENDATION_TRENDS). - history only supports daily intervals; the interval parameter is ignored and daily bars are always returned. The period parameter is used to compute a date cutoff that filters the returned bars. - Fundamentals, search, options, and currency endpoints are not implemented. With yahoo_fallback: true, these fall back to Yahoo.

TwelveData (paid): - Implements quote (via /quote), history (via /time_series), and fundamentals (via /statistics). - /statistics requires a paid TwelveData plan; the free tier returns an error which triggers Yahoo fallback if enabled. - history supports the interval parameter mapped to TwelveData intervals (1day, 1week, 1month). - London Stock Exchange symbols with a .L suffix are automatically routed to the LSE exchange. - Search, options, recommendations, and currency endpoints are not implemented. With yahoo_fallback: true, these fall back to Yahoo.

Error Handling

All finance tools return normalized errors via the `FinanceError` enum (defined in `crates/ragent-tools-extended/src/finance_error.rs`).

Error Variant	Meaning
<code>SymbolNotFound</code>	The requested ticker could not be resolved by the provider.
<code>RateLimit</code>	The provider rate-limited the request. Includes an optional <code>retry_after</code> hint in seconds.
<code>ProviderFailure</code>	A provider-side failure that is not a rate limit or parse error. The message identifies the failing endpoint.
<code>ParseFailure</code>	The provider returned data that could not be parsed into the normalized model.
<code>ConfigError</code>	The configuration for the requested provider is missing or invalid (e.g. empty API key).

When a tool encounters an error, it returns the error message as the tool output. The agent can then decide whether to retry, fall back, or report the error to the user.

HTTP API

All finance tools are also available via the HTTP server's tool execution endpoint. The server runs on port 9100 by default (`ragent serve --port 9100`). Each tool can be invoked by name with its JSON input:

```
# Get a stock quote via HTTP
curl -s -X POST http://localhost:9100/tools/stock_quote \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"symbol": "AAPL"}'
```

```
# Get historical prices
curl -s -X POST http://localhost:9100/tools/stock_history \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"symbol": "MSFT", "interval": "1d", "period": "3mo"}'

# Get a currency rate
curl -s -X POST http://localhost:9100/tools/currency_rate \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"base": "USD", "quote": "JPY"}'
```

Practical Examples

Example 1: Portfolio Price Check

Ask the agent to check the latest prices for a watchlist:

Check the current prices for AAPL, MSFT, GOOGL, AMZN, and NVDA and give me a summary table.

The agent will call `stock_quote` five times (with 60-second caching) and present a summary.

Example 2: Compare Two Stocks

Compare TSLA and F over the past 3 months. Show me their daily closing prices and tell me which performed better.

The agent will call `stock_history` for both symbols with `{"interval": "1d", "period": "3mo"}`, compute the returns, and summarize.

Example 3: Find a Ticker

I want to invest in a company called "Palantir". What is its ticker symbol and current price?

The agent will call `stock_search` with `{"query": "Palantir"}` to find the ticker PLTR, then call `stock_quote` with `{"symbol": "PLTR"}`.

Example 4: Currency Conversion

I have 5000 USD. How many EUR will I get at the current rate?

The agent will call `currency_rate` with `{"base": "USD", "quote": "EUR"}`, get the rate, and compute $5000 * \text{rate}$.

Example 5: Fundamentals Screen

Show me the fundamentals for JPM and BAC. Which has a lower P/E ratio?

The agent will call `stock_fundamentals` for both symbols and compare the `trailing_pe` field.

Example 6: Options Analysis

Show me the call options for SPY at the nearest expiration with strike prices between 450 and 460.

The agent will call `stock_options` with `{"symbol": "SPY"}` and filter the results by `kind == "Call"` and `450 <= strike <= 460`.

Example 7: Analyst Sentiment

What is the analyst consensus for NVDA? Are more analysts saying buy or sell?

The agent will call `stock_recommendations` with `{"symbol": "NVDA"}` and sum the `strong_buy + buy` counts versus `sell + strong_sell`.

Example 8: FX Trend Analysis

Show me the weekly GBP/USD exchange rate for the past 6 months. Is the trend up or down?

The agent will call `currency_history` with `{"base": "GBP", "quote": "USD", "interval": "1wk", "period": "6mo"}` and analyze the closing prices.

Caching and Rate Limiting

Quote Cache

The `stock_quote` tool uses an in-memory `QuoteCache` with a default 60-second TTL. Quotes are keyed by (provider, symbol) and evicted lazily on read when they expire. This prevents redundant API calls when an agent asks for the same symbol multiple times in quick succession. The cache is per-process and does not persist across restarts.

Rate Limiter

The `RateLimiter` (defined in `crates/ragent-tools-extended/src/finance/rate_limit.rs`) tracks per-provider rate-limit state. When a provider returns a 429 or similar rate-limit response, the limiter records the event and applies exponential backoff (up to `MAX_BACKOFF_SECONDS`). Subsequent calls to the same provider are short-circuited with a `RateLimit` error until the backoff window expires.

Throttle

The `min_call_interval_seconds` config field (default 5 seconds) adds a global minimum interval between any two finance API calls, shared across all providers. The `wait_for_min_interval` helper (in `crates/ragent-tools-extended/src/finance/throttle.rs`) blocks until the minimum interval has elapsed since the last call. The Yahoo adapter also has an optional `Throttle` struct that enforces a `requests_per_minute` target with precise async sleeping.

Troubleshooting

“rate limit hit for provider yahoo”

Yahoo Finance imposes rate limits on unauthenticated scraping. If you hit this frequently, either: - Set `"requests_per_minute": 30` (or lower) in the finance config to throttle client-side. - Set `"user_agent": "MyApp/1.0"` to use a custom User-Agent. - Switch to a paid provider (Alpha Vantage or TwelveData).

“symbol not found: XYZ”

The provider could not resolve the ticker. Check the symbol spelling. For non-US exchanges, Yahoo uses suffix notation (e.g. `VOD.L` for Vodafone on the LSE, `BHP.AX` for BHP on the ASX). Use `stock_search` to find the correct symbol.

“not implemented for paid provider”

The configured paid provider does not implement the requested endpoint. Enable Yahoo fallback with `"yahoo_fallback": true` in the finance config, or switch back to `"provider": "yahoo"` for full coverage.

“finance configuration error: Alpha Vantage API key missing”

You set `"provider": "alpha_vantage"` but did not provide an `api_key`. Add the `api_key` field to the finance config block.

TwelveData fundamentals fail on free tier

TwelveData's `/statistics` endpoint requires a paid plan. On the free tier, `stock_fundamentals` will return a `ProviderFailure`. Enable `"yahoo_fallback": true` to fall back to Yahoo for fundamentals while keeping TwelveData for quotes and history.